

Morphic Studio Software Architecture Specification

Document 39 — Frontend Architecture Specification

Version: 1.0

Status: Approved

1. Purpose

This document defines the architecture of the Morphic Studio frontend.

It specifies how the user interface is structured, how data flows through the application, how state is managed, and how components are organized to support a scalable AI-native creative platform.

The frontend must remain modular, performant, accessible, responsive, and maintainable while supporting future expansion without architectural redesign.

2. Frontend Philosophy

The frontend follows seven principles:

2.1 Feature-First Architecture

Features are organized around user capabilities rather than technical categories.

Examples:

- Story Workspace
- Story Bible
- Character Manager
- World Builder
- Production Studio
- Export Center

Each feature owns:

- Components
 - Hooks
 - API interactions
 - Local state
 - Tests
 - Documentation
-

2.2 Composition Over Inheritance

Complex interfaces should be built from small reusable components.

Examples:

Button

↓

Card

↓

Panel

↓

Workspace Layout

↓

Story Workspace

2.3 Server Components First

Whenever appropriate, use React Server Components to reduce client-side JavaScript.

Interactive experiences should use Client Components only where necessary.

2.4 Accessibility by Default

Every UI component must support:

Keyboard navigation

Screen readers

Visible focus

Proper semantic HTML

ARIA where required

2.5 Responsive First

Every feature must function correctly on:

Desktop

Laptop

Tablet

Mobile

Layouts should adapt without reducing usability.

2.6 Progressive Enhancement

Core functionality should remain usable even if advanced browser capabilities are unavailable.

2.7 AI-Enhanced Interface

AI is integrated into workflows rather than isolated in a chat window.

Recommendations appear contextually.

3. Technology Stack

Framework:

Next.js (App Router)

Language:

TypeScript

UI Library:

React

Styling:

Tailwind CSS

Component Library:

shadcn/ui (customized)

Animation:

Motion

Icons:

Lucide

Forms:

React Hook Form + Zod

State:

Zustand + TanStack Query

Tables:

TanStack Table

Charts:

Tremor or Recharts

Markdown:

MDX

Rich Text:

Tiptap

4. Folder Structure

apps/web/

app/

components/

features/

hooks/

contexts/

providers/

styles/

lib/

services/

types/

config/

constants/

public/

tests/

Each directory has one clearly defined responsibility.

5. Routing Strategy

Use Next.js App Router.

Examples:

/

Landing

/login

/dashboard

/projects

/projects/[id]

/story

/story-bible

/characters

/world

/assets

/storyboard

/comic

/animation

/export

/settings

/help

Route groups should separate:

Public

Authenticated

Admin

API

Error Pages

6. Layout Architecture

Define reusable layouts.

Examples:

Marketing Layout

Authentication Layout

Workspace Layout

Settings Layout

Admin Layout

Every page inherits from a layout instead of recreating navigation.

7. Component Hierarchy

Components are divided into four levels.

Primitive Components

↓

Composite Components

↓

Feature Components

↓

Application Pages

This prevents duplicated logic and encourages reuse.

8. State Management

State is divided into:

Server State

Client UI State

Editor State

Authentication State

Workspace State

AI Task State

Avoid global state unless necessary.

Use:

TanStack Query for server data.

Zustand for UI state.

React Context only for truly global concerns.

9. Design System Integration

All UI must consume the shared Design System.

No page should define independent colors, typography, spacing, or component styles.

Every component must reference design tokens.

10. Theme Engine

Support:

Light Theme

Dark Theme

System Theme

Future themes should be added without changing component logic.

11. Performance Strategy

Implement:

Route-based code splitting

Dynamic imports

Image optimization

Virtualized lists

Lazy loading

Optimistic UI updates

Streaming

Suspense boundaries

Background prefetching

Aim for fast perceived performance rather than simply minimizing bundle size.

12. Accessibility Standards

Target WCAG 2.2 AA compliance.

Requirements include:

Keyboard navigation

High contrast support

Accessible forms

Focus management

Reduced motion support

Screen reader compatibility

Large touch targets

Accessible error messaging

13. Offline Strategy

Support graceful degradation.

Features:

Cached assets

Offline project viewing

Background synchronization

Reconnect detection

Conflict handling

The application should remain reliable under unstable network conditions.

14. Error Handling

Implement:

404

403

500

Network failures

AI provider failures

Validation errors

Timeout handling

Every error should provide clear recovery guidance.

15. Internationalization

Design for multilingual support from Version 1.

Requirements:

Externalized strings

RTL readiness

Locale-aware formatting

Language switching

No hardcoded UI text inside components.

16. Testing Strategy

Frontend testing includes:

Unit tests

Component tests

Integration tests

Accessibility tests

Visual regression tests

End-to-end tests

Testing should be automated within CI.

17. Acceptance Criteria

The frontend architecture is successful if:

Features remain modular.

Developers can locate code quickly.

AI coding agents understand project organization.

Pages share reusable components.

Performance remains consistent as features expand.

Accessibility is maintained.

New features integrate without restructuring.

Conclusion

The Morphic Studio frontend architecture establishes a scalable, modular, AI-friendly foundation capable of supporting a world-class creative platform. By combining feature-first organization, shared design systems, modern React architecture, and accessibility-first development, the frontend is prepared for long-term growth while remaining maintainable for both human developers and AI coding agents.